Ensayo de entrevista: streaming + spec-driven development

Este fichero es para practicar en voz alta, no para leer. La idea es taparte `streaming-mastery.md` y `spec-driven-development-mastery.md`, responder cada pregunta como si estuvieras delante de un entrevistador, y solo después comparar con la respuesta modelo. Si tu respuesta necesitó más de dos frases para llegar al punto, está bien para entrenar, pero en la entrevista real, apunta a la versión corta primero y deja que te pidan más detalle.

Cada respuesta modelo termina en un ejemplo real de este proyecto. Sin ejemplo, la respuesta suena a manual. Con ejemplo, suena a alguien que lo ha hecho.

Streaming

1. ¿Por qué un sistema de streaming necesita watermarks si ya tiene timestamps de evento?

Porque un timestamp por sí solo no te dice cuándo puedes dejar de esperar más datos para una ventana de tiempo, solo te dice cuándo ocurrió un evento concreto, no si van a llegar más eventos anteriores a ese momento. El watermark es la decisión explícita del sistema de "asumo que ya he visto todo lo que tiene event-time ≤ T", con margen de seguridad. Sin esa decisión, ninguna ventana podría cerrarse nunca con certeza.

2. Tu job deja de avanzar solo para un segmento de datos. ¿Cómo depuras esto en los primeros cinco minutos?

Primero comparo el paralelismo configurado del operador de entrada contra el número real de particiones del topic. Si el paralelismo es mayor, ya tengo la causa: subtasks sin partición asignada, congelados desde el arranque, sin relación con el tráfico. Si el paralelismo cuadra, entonces sospecho de una partición con tráfico real pero temporalmente inactiva, y reviso si `with_idleness` está configurado y con qué umbral. En este proyecto el ejemplo real es `booking-events`, con una sola partición: si no hubiera forzado `parallelism=1` en ese operador, tres de los cuatro subtasks por defecto se habrían quedado sin datos para siempre.

3. Diseña un acumulador para una media ponderada por streaming. ¿Qué orden sigues y qué prueba primero?

Escribo la fórmula final con números concretos antes que nada, no la abstracción. Identifico qué se acumula de forma independiente, nunca "una media de medias", siempre sumas crudas: `Σ(valor · peso)` y `Σ(peso)` por separado, dividiendo solo una vez en `get_result()`. Reviso el elemento identidad de cada acumulador (`0` para sumas, no una media a medio calcular). Y pruebo el caso de un solo evento y el de cero eventos antes de darlo por bueno. El error que evito con esto: una media incremental tipo `(media_anterior + nuevo) / n` da resultados matemáticamente incorrectos. Con `[100, 100, 300]` da `133.3` en vez de `166.7`, porque trata una cantidad ya dividida como si pesara lo mismo que un valor individual sin dividir.

4. ¿Cuándo usarías `min` / `max` con `0` como valor inicial, y cuándo no?

Casi nunca con `0`. `0` solo es seguro si conoces y confías en una restricción de dominio actual (un campo que nunca es negativo) que podría cambiar en el futuro. `-infinito` para un máximo y `+infinito` para un mínimo son correctos siempre, sin depender de ninguna restricción externa. El motivo concreto: si el

mismo patrón de acumulador se reutiliza mañana para un campo que sí admite negativos, `max(0, ...)` reportaría `0` como "máximo" cuando en realidad nunca ocurrió ese valor.

5. Explica qué pasa con un evento que llega tarde, en las tres capas posibles.

Dentro del margen de desorden esperado, el watermark simplemente todavía no ha decidido cerrar la ventana, así que el evento entra con normalidad. Dentro de un margen de tolerancia explícito más allá de eso, `allowedLateness`, la ventana ya se disparó, pero se vuelve a disparar incluyendo el nuevo dato; si el sink escribe de forma idempotente por clave primaria, la segunda escritura simplemente sobreescribe a la primera, sin necesitar lógica de merge. Fuera de ese segundo margen, el evento nunca se descarta en silencio, va a un side output, como mínimo logueado, idealmente auditable. En este proyecto ese tercer nivel estuvo completamente ausente del código durante buena parte de su desarrollo, pese a que el diseño del TTL de deduplicación ya asumía que iba a existir; se detectó buscando literalmente las palabras clave (`allowed_lateness`, `OutputTag`) en el código y no encontrando ninguna.

6. ¿Cómo deduplicarías eventos de un pipeline CDC sin depender de campos que rellena otra aplicación?

Con estado propio, con clave: un `ValueState[bool]` indexado por el identificador de negocio del evento (no por un campo auxiliar como `updated_at`), con TTL igual a ventana más lateness, lo bastante largo para cubrir un redelivery o restart reciente, pero no más, porque un resnapshot completo con timestamps antiguos ya se descarta gratis por el `allowedLateness`. Evito depender de campos como `updated_at IS NULL`, porque son un contrato implícito con el comportamiento de otra aplicación (un trigger que no controlo) y se rompen sin aviso. En este proyecto, un resnapshot de Debezium reemite una fila nunca editada con `updated_at = NULL`, indistinguible de un insert genuino si esa fuera la única señal.

7. ¿Por qué RocksDB como state backend no es un cuello de botella si cada acceso es más lento que en memoria?

Cuatro razones que se combinan: la carga de trabajo de un operador con estado es casi toda escritura, el patrón exacto para el que un LSM-Tree está optimizado; el block cache mantiene en RAM las claves más consultadas; el acceso es casi siempre un point lookup por clave exacta, no un rango, con Bloom filters que evitan lecturas en vano; y no hay latencia de red porque RocksDB corre embebido en el mismo proceso. Ninguna por separado explica el resultado, es la combinación de las cuatro.

8. Diferencia entre checkpoint y savepoint, y qué pasa con la deduplicación al reiniciar desde cualquiera de los dos.

El checkpoint es automático y periódico, pensado para recuperación de fallos. El savepoint es manual y deliberado, disparado antes de una operación planeada como un upgrade o un cambio de paralelismo. La garantía de fondo (exactly-once, basada en barreras) es la misma en ambos. Al reiniciar desde cualquiera de los dos, Kafka reenvía eventos desde el último offset confirmado, algunos de los cuales ya se procesaron antes del reinicio. Si el TTL de deduplicación cubre esa ventana de reprocesamiento posible, el reinicio es seguro sin contar nada dos veces; si el retention de Kafka fuera menor que ese TTL, hay un agujero de recuperación, pero eso es un problema de retention, no de la lógica de dedup en sí.

9. ¿Por qué existe Async I/O como mecanismo aparte, en vez de una llamada normal dentro de un `map()`?

Porque un operador normal procesa un evento y no puede empezar el siguiente hasta que termina. Si esa llamada es una petición de red, el throughput queda limitado por la latencia de esa llamada, evento a evento, sin solapamiento. `AsyncDataStream.unordered_wait` deja varias llamadas en vuelo a la vez hasta un límite de `capacity`, sin bloquear el operador mientras esperan respuesta. En este proyecto esto enriquece cada reserva creada con un lookup a DynamoDB para calcular su profit, y `capacity` es literalmente el

punto de backpressure del pipeline: demasiado bajo, el job va más lento de lo necesario; demasiado alto, satura la tabla remota.

10. Cuéntame un incidente real de streaming que hayas depurado.

Ten preparado el de `booking-events` con una sola partición real, completo: síntoma, el watermark del operador de bookings nunca avanzaba pese a tráfico normal; causa, paralelismo por defecto de 4 contra 1 sola partición real, tres subtasks sin datos desde el arranque; fix, `set_parallelism(1)` en ese operador concreto, no un `with_idleness` que habría disimulado el síntoma sin arreglar la causa. Ten un segundo de reserva: la variable de entorno de DynamoDB ausente que hacía que el job hablara con AWS real con credenciales falsas en vez de con LocalStack, encontrada leyendo la traza exacta del error, no adivinando.

Spec-driven development

11. ¿Cuál es la diferencia real entre una spec de fase y un ADR?

Una spec de fase describe qué se va a construir y cómo se sabrá que está bien construido, tiene que sobrevivir sin quien la escribió delante, con criterios de aceptación verificables por un tercero. Un ADR documenta una bifurcación puntual, y lo que más importa en él no es la opción elegida sino la alternativa rechazada y por qué, con el mismo nivel de detalle. Escribo un ADR cuando la decisión trasciende una sola fase y alguien en el futuro podría proponer deshacerla sin conocer el contexto ya descartado. Por ejemplo, `ADR-0008` de este proyecto no es "cómo consumo Kinesis en esta fase", es "por qué no existe ya un conector nativo de Kinesis en Flink 2.x, y qué se hizo en su lugar".

12. ¿Cómo verificas que una implementación cumple su spec, más allá de que los tests pasen?

Con fault injection contra el sistema real, no solo contra mocks: capturar un evento real de Kafka y republicarlo para simular una redelivery, cruzando el resultado contra la base de datos fuente para distinguir un duplicado genuino de un evento nuevo coincidente. Esto encontró tres bugs reales en este proyecto que ningún test unitario tocaba: una variable de entorno ausente que hacía que el job llamara a AWS real con credenciales falsas, un contenedor con una imagen Docker obsoleta, y dos criterios de aceptación completos sin implementar pese a estar escritos en la spec desde el principio.

13. Un test unitario pasa, pero el criterio de aceptación no se cumple. ¿Cómo es posible, y cómo lo evitas?

Un test solo prueba lo que a alguien se le ocurrió probar. Un AC completo puede quedar ausente del código y convivir semanas con una suite en verde si ningún test ejercita esa ruta concreta. Pasó de verdad en este proyecto con dos criterios de aceptación de manejo de datos tardíos, detectados no por un test que fallara sino por buscar en el código las palabras clave que el AC exigía y no encontrar ninguna coincidencia. Lo evito tratando "los tests pasan" y "cumple la spec" como dos afirmaciones distintas que hay que verificar por separado, nunca como sinónimos.

14. Revisas una spec escrita por otra persona antes de dar el visto bueno para construir sobre ella. ¿Qué miras primero?

Si sus criterios de aceptación son verificables por alguien que no sea su autor, una condición concreta, no una esperanza en prosa. Y si compara la spec contra lo ya existente (en una auditoría de gap analysis), si distingue con claridad un gap de cobertura funcional (falta una pieza, es trabajo pendiente) de un gap de arquitectura (el diseño elegido no soporta lo pedido sin rediseñar): son problemas de naturaleza distinta y confundirlos hace que el informe suene más grave o más leve de lo que realmente es.

15. Dame un ejemplo de algo que una PoC dejó simplificado a propósito, y qué tendría que pasar para que dejara de ser aceptable.

El versionado de estrategias de pricing: en una PoC es aceptable no tenerlo, porque nadie audita decisiones históricas todavía. Deja de ser aceptable en cuanto hay un cliente real y alguien necesita poder explicar, meses después, qué regla estaba activa en el momento de una decisión concreta de precio; eso pasa de ser una comodidad de ingeniería a un requisito de auditabilidad legal/contractual. Contraste: los datos de mercado sintéticos son sobre todo un problema de calidad del producto (¿la recomendación es útil de verdad?), con una capa añadida de riesgo legal si se sustituyeran por scraping real sin revisar los términos de servicio del sitio de origen. Son dos tipos de "no negociable" distintos, con plazos y urgencias distintas.

Antes de la entrevista: la prueba de fuego

Cierra los tres ficheros y responde estas dos preguntas de memoria, sin mirar nada:

1. Explica, con un ejemplo numérico, por qué una media incremental está mal.
2. Explica la diferencia entre un checkpoint y un savepoint, y qué garantiza cada uno en el contexto de un reinicio.

Si dudas en cualquiera de las dos, vuelve a `streaming-mastery.md`, no a este fichero. Este es solo para practicar la forma de la respuesta, no para aprender el contenido por primera vez.